MoveInSync Rating Project: Complete Study Guide

Oct 12, 2026 · @Abhishek Jain

The project in 60 seconds

I found out from public data why the MoveInSync employee app's Play Store ratings suffer, ranked what to fix first, and built a tool that answers "How are our Play Store ratings?" again every month.

The Play listing shows 4.6 stars, but one in four recent text reviews is a 1-star review. I collected 24,314 public reviews, kept the last year (2,856 reviews, of which 846 are rated 1-3 stars), and sorted the complaints into 26 types with a rule-based classifier. I then measured how accurate that classifier is, including a check by a human (me) on 30 reviews.

The biggest single complaint group is the app crashing, not opening or loading slowly: 150 reviews, about 18% of the low-star pool. A stability fix is the only backlog item that stays in the top three when I shake the scores 5,000 times. Everything after it is a statistical tie.

Every claim is labelled as a fact, an estimate or a hypothesis. Public reviews are a biased sample, and I have not seen MoveInSync's own data, so I make no promise about star lift.

The problem and what the case study asked for

MoveInSync is a corporate commute platform: employees book cabs and shuttles through the employee app (`com.moveinsync.ets` on Google Play). The case study, for a Forward Deployed Engineer (Intern-to-FTE) role, asks you to work out why the app's ratings are low using only public data, and to deliver four things.

The case asked for	Where it is in the repo
Why the ratings are low	`docs/phase3_findings.md`, `docs/phase4_root_cause_register.md`
What to fix first, detailed enough for Product and Engineering to start	`docs/EXECUTIVE_RECOMMENDATION.md`, `docs/engineering_tickets.md`

The case asked for	Where it is in the repo
The method and tools used	`docs/methodology.md`, `docs/source_register.md`
A repeatable process, because "How are our Play Store ratings?" will be asked again	The `rating_intel` package and `scripts/run_all.ps1`

The rules I set myself, and should be ready to repeat:

- **Scope:** the employee app only, not the driver, Field or Partner apps.
- **Free tools:** Python and open-source libraries. No paid API.
- **No invented data:** facts, estimates and hypotheses are labelled separately.
- **No dishonest rating optimisation:** no incentives, no selective prompts, no hiding negative reviews.

The starting puzzle: the listing average is 4.6, yet text reviews average 3.8. They are different groups of people. Only about one in five raters writes text, and unhappy users write more often, so reviews are a biased sample.

What we did, phase by phase

The work ran in nine phases, each checked before the next. For every phase, learn the "why" as well as the "what", because interviewers ask why.

Phase 1: Baseline (where do we stand?)

- Read the public Play listing: score 4.6377 (shown as 4.6), about 114,857 ratings.
- Rating histogram, 1 to 5 stars: about 7,335 / 837 / 1,230 / 7,294 / 98,153.
- Key finding: 1-star ratings account for 70.5% of the shortfall from a perfect 5.0. The low score is mostly a 1-star problem, not a general disappointment.
- Why: you cannot say what is wrong before you know what "low" means and who causes it.

Phase 2: Collection (get the data, and trust it)

- Tool: `google-play-scraper` 1.2.7, language `en`, country `in`, newest first. I collected 24,314 text reviews from 2014 to 2026.

- Checks: no duplicates, no rejected rows, and 24,314 rows against the listing's 23,617 text-review count (103%).
- Privacy: reviewer names and photos are not stored.
- Why: a scraper can silently stop early. Comparing with the listing count catches that.

Phase 3: Complaint structure (what do people complain about?)

- Read 502 development reviews by hand and derived 26 complaint categories from what people actually wrote (for example login/session, OTP, crash/loading, tracking, booking failure).
- Built a rule-based (keyword/regex) classifier. One review can carry several categories.
- Validated it: round 1 on 120 reviews (used to improve rules, so optimistic); round 2 on 100 fresh reviews with rules frozen: precision 0.76, recall 0.65, F1 0.70.
- Why rules and not an LLM: free, repeatable, and every decision can be inspected. An optional local-LLM layer exists but is off by default.

Phase 4: Root causes (why do these happen?)

- A register of 11 hypotheses (R1 to R11). Each names the symptom, the evidence, the likely cause, and a test that would confirm or reject it.
- Used public Help Center and Android documentation (sources S13 to S21): no-show rules, the communication feature, Android vitals thresholds, Play Integrity.
- Why: public reviews show symptoms, not causes. Calling a guess a fact would be dishonest, so each cause is a hypothesis with a test.

Phase 5: Prioritisation (what first?)

- Score = Reach x Severity x Confidence x Controllability / Effort. Reach is measured (share of low-star reviews hit). The other factors are judgement and are labelled that way.
- Sensitivity test: shake the judgement scores 5,000 times and see whether the ranking holds. Only stability (B1) held, at 79% of trials in the top three.
- Why: a ranked list with fake precision is worse than none. The test shows which conclusions are robust.

Phase 6: Measurement plan (how will we know a fix worked?)

- Primary measures are internal telemetry (crash rate, OTP success, and so on). Public review complaint rates are a slow, noisy secondary signal.
- Reason: with about 240 text reviews a month, only a swing of roughly 11 points in the 1-star share is detectable. Reviews cannot judge a single release.

Phase 7: The reusable tool (MoveInSync Rating Intelligence)

- Python package `rating_intel` with commands `clean`, `status`, `analyze` and `compare`. One PowerShell script (`run_all.ps1`) runs the full monthly refresh.
- 85 automated tests, including a check that the numbers quoted in the recommendation still match the data.
- Why: the case says the question will be asked repeatedly, so the process has to be repeatable, not a one-off report.

Phase 8: Submission package

- README, executive recommendation, engineering tickets, methodology, a short note for the form, MIT licence, CI workflow, public GitHub repo.

Phase 9: Adversarial review (attack your own work)

- An independent reviewer, given only documents and code, found real defects. I fixed them: a volume artifact (a "June 2026 cluster" that disappears once you account for the number of reviews that month), rating arithmetic that used the rounded 4.6, an inflated login-burst weight, and several unsourced claims.
- Why this matters in an interview: you can say what was wrong with your first version and how you found out.

Last step: human check of the labels

- The validation labels were first made by the AI. I then labelled 30 reviews myself without seeing them: kappa 0.84 against the AI labels, F1 0.67 against the rule classifier.

Key numbers cheat sheet

Memorise the bold ones. Everything comes from the snapshot taken on 2026-10-10 (window 2025-10-10 to 2026-10-09).

What	Number
Play listing score	**4.6377** (shown 4.6) from about **114,857** ratings
Share of the shortfall from 5.0 caused by 1-star ratings	**70.5%**
Text reviews collected, all years	**24,314** (listing says 23,617)
Reviews in the one-year window	**2,856**
Complaint pool (rated 1-3 stars)	**846** (772 rated 1-2 stars)
Text review mean and 1-star share, window	3.83 stars, **24.0%** one-star
Last 90 days vs previous 90	3.80 vs 3.83 stars, 1-star share 25.0% vs 25.0%: **no distinguishable change** (p = 0.73 and 1.00)
Complaint categories	**26**
Classifier accuracy (frozen test, 100 reviews)	precision **0.76**, recall **0.65**, F1 **0.70**
Human check on 30 reviews	kappa **0.84** vs AI labels; F1 **0.67** vs the rule classifier
Largest complaint group: crash, not opening, slow loading	**150** reviews, **17.7%** of the pool (19.7% in the recent half)
Stability fix (B1) in the top three across 5,000 sensitivity trials	**79%**
Detectable change with one month of reviews (about 240)	about **11 points** in 1-star share
Detectable change with a quarter of reviews	about **6 points**
To display 4.7 (if the display is the plain rounded mean, unverified)	about **4,000** extra 5-star ratings, or about **350** 1-star ratings turning into 5-star
To reach a true 4.7	about **24,000** extra 5-star ratings
Developer reply rate to reviews	97.8% in 2023, 4.3% in 2025, 0.8% in 2026 so far

What	Number
Only strong statistical spike	install error, July and August 2026 (5 vs 0.35 expected; 9 vs 1.2 expected)
Automated tests	**85** passing (69 without the data snapshot)

Concepts explained simply

You do not need the maths, only the idea and one example.

- **Selection bias.** Only some users write reviews, and angry users write more often. So reviews are not a fair picture of all users. This is why text reviews average 3.8 while all ratings average 4.6.
- **Precision.** Of the reviews my classifier labelled "login problem", how many really were? 0.76 means about three in four.
- **Recall.** Of all reviews that really were login problems, how many did the classifier find? 0.65 means about two in three. So its counts are approximate, and in total it under-counts by about 14%.
- **F1.** One number that blends precision and recall. 0.70 is decent for a keyword-rule system.
- **Cohen's kappa.** How much two labellers agree beyond what luck would give. Above about 0.6 is substantial. My 0.84 against the AI labels is strong.
- **Wilson confidence interval.** A range that is likely to contain the true share, given a small sample. For example, 25% one-star with n = 721 means somewhere roughly 22% to 28%. Wilson behaves better than the naive formula when counts are small.
- **Two-proportion z-test and p-value.** Is the last 90 days different from the 90 before? A p-value of 0.73 means a difference this size is very easy to get by chance, so we say no change.
- **Poisson spike detection.** Complaint counts bounce around randomly. I compute the expected count for a month and ask how unlikely the observed count is. Only install errors in July and August 2026 were unusual enough.
- **Bonferroni correction.** If you test 230 things, a few will look "significant" by luck. So you demand a much stricter threshold. This is what removed the fake "June cluster".
- **Sensitivity test.** My severity, confidence and effort scores are judgement. I randomly nudged them 5,000 times and watched whether the ranking changed. Only the first place was stable.

- **Detectable effect.** With few reviews per month, small improvements are invisible in the star average. This is why fixes must be measured with internal telemetry, not Play stars.
- **Rule-based vs LLM classifier.** Rules are free, deterministic and explainable. An LLM would read better but costs money or compute and varies between runs. The repo supports a local model as an option, off by default.

Map of the repo

Repo: github.com/Abhijain01/moveinsync-rating-intelligence. Know what each part is for, so you can open it in an interview and point.

Path	What it is
`README.md`	Entry point: what the project is, how to run it, results at a glance
`docs/EXECUTIVE_RECOMMENDATION.md`	The answer for Product and Engineering: what to fix first and why
`docs/engineering_tickets.md`	Ticket-level specs: evidence, scope, acceptance criteria, telemetry, guardrails
`docs/phase5_backlog.md`	The scoring model, the 5,000-trial sensitivity result, the ranked backlog
`docs/phase4_root_cause_register.md`	The 11 hypotheses R1 to R11 with the test for each
`docs/phase6_measurement_plan.md`	How to judge fixes, what reviews can and cannot detect
`docs/methodology.md`	Method, tools, validation, honest limits, the human check
`docs/source_register.md`	Every source used (S1 to S21), so claims are traceable
`docs/SUBMISSION_NOTE.md`	The short text pasted into the submission form
`rating_intel/`	The reusable Python package (the tool)

Path	What it is
`rating_intel/clean.py`	Validates raw reviews, drops duplicates, writes a quality report
`rating_intel/classify.py`, `taxonomy.py`	The 26 categories and the keyword rules that apply them
`rating_intel/stats.py`	Wilson intervals, z-tests, Poisson helpers, detectable-effect maths
`rating_intel/trends.py`	Monthly summaries and spike detection
`rating_intel/status.py`	The "How are our ratings?" headline (last 90 days vs previous 90)
`rating_intel/pipeline.py`, `cli.py`	Runs the full analysis and exposes the commands
`rating_intel/compare.py`	Compares two snapshots (what is new since last month)
`rating_intel/llm.py`	Optional local-LLM labelling, off by default, local Ollama only
`scripts/run_all.ps1`	One command for the monthly refresh on Windows
`scripts/ingest.py`	Collects reviews from Google Play
`scripts/prioritize.py`	Computes the scored backlog and the sensitivity test
`scripts/human_audit.py`	Builds the blind labelling sheet and scores human vs AI agreement
`tests/`	85 automated tests, including a check that quoted numbers match the data
`reports/phase3`, `reports/phase5`	Saved evidence: validation, backlog scores, human agreement

The raw data (`data/`) is not in the repo, on purpose. It is rebuilt with `ingest.py`.

What to fix first and why

Start with a one-to-two-week instrumentation gate (B0), then build stability resilience (B1). Only B1 is a robust first place. Everything below it is statistically a tie, so do not defend the exact order.

Rank	Item	In plain words	Reach in low-star reviews (recent half)
0	B0 Instrumentation gate	Pull crash and ANR data by version and device, the OTP funnel, location ping gaps, no-show events, and Play Console ratings by version. Confirms or rejects each hypothesis.	not scored
1	B1 Stability resilience	When the network is slow or fails, show the last known trip with its age, give timeouts and a visible Retry. Helps whichever backend cause sits behind slow loading. Does not fix "will not open" or crashes: those need B0 data.	19.7%
2	B8 Skippable trip rating	Let people skip the rating, ask for consent once. Never steer users to the Play Store. Cheap, but its effect on stars is unmeasurable.	5.4%
3	B3 Authentication clarity	Separate messages for "not registered" and "OTP delayed", resend on another channel, no logout during an active trip. Chosen first among the tie by judgement.	9.2%

Rank	Item	In plain words	Reach in low-star reviews (recent half)
4	B4 Tracking honesty	Show "last updated hh:mm", stale-pin state, smoothed ETA.	10.3%
5	B6 Booking rule explanations	Show the rule behind every rejection.	9.9%
6	B10 Notifications and pickup-time changes	Fewer, clearer messages.	4.5%
7	B7 In-app helpdesk and driver contact	Assumption to confirm: the Help Center describes admin-to-user messaging only.	8.2%
8	B13 Boarding and safety-check flow	QR scan and boarding verification problems.	5.2%
9	B2 Install and launch error	Diagnosis spike. The only statistically strong spike (July and August 2026).	6.0%
10	B11 UI usability triage	Work through UI complaints with Design.	8.6%
11	B5 No-show transparency	Diagnosis spike.	2.1%
-	B12 Reply to low-star reviews	Start now. Replies name a fix or where to get help, never ask for a re-rating.	not scored
-	B9 Operations	Routing and vehicles: hand to Customer Success per client, not a mobile ticket.	15.5%

Why B1 first. Largest single group, and it is the only item in the top three in most sensitivity trials (79%).

Why B9 is last even though its reach is high. The app team cannot control it. The formula includes controllability.

Why B0 first, before any build. Public reviews show symptoms, not causes. Without crash and funnel data, building would be guessing.

How to actually improve the rating: the honest approach

The right way to raise a rating is to fix what makes people give 1 star, then measure it with the company's own data. Tricks that move stars without fixing the experience are dishonest and risky, and this project rules them out.

The logic in four steps

1. **Find where the 1-star ratings come from.** 70.5% of the shortfall from 5.0 is 1-star ratings. Complaint themes: crash and loading, login and OTP, tracking, booking, and forced rating prompts.
2. **Measure causes with internal data first (B0).** Android vitals by version and device, OTP success, location staleness, no-show events. Google's "bad behaviour" thresholds are 1.09% crash and 0.47% ANR of daily users, 8% for a single phone model.
3. **Fix in this order:** stability resilience (B1), authentication clarity (B3), the forced rating prompt (B8), then tracking and booking explanations. Release through staged rollout and compare the new version with the old on the same days.
4. **Judge success by the targeted failure, not by the star average.** Did crashes, OTP failures and forced logouts fall? The star average moves slowly: about 0.000035 per avoided 1-star rating on the all-time average.

Honest levers that help, with limits

- **Fix the cause of 1-star reviews.** The main lever.
- **Reply to low-star reviews.** Developer replies fell from 97.8% in 2023 to 0.8% in 2026 so far. Replies should name a fix or where to get help, never ask for a re-rating. The effect on stars is unmeasurable from public data.
- **Do not make users rate in anger.** One review says the repeated rating prompt dragged their rating from 5 to 3 stars. Make the trip rating skippable and ask for consent once.
- **Use the official Play in-app review prompt neutrally,** shown to everyone at a natural moment, not only to happy users. Google's guidance says not to ask opinion questions before the prompt.

What not to do (and say so in the interview)

- No incentives or rewards for ratings.
- No selective prompts that only reach happy users ("review gating").

- No steering users to 5 stars or hiding or burying negative reviews.
- No promise of a star lift without a defensible model.

How much would it take? If the display is the plain rounded mean (unverified), showing 4.7 needs about 4,000 extra 5-star ratings, or about 350 one-star ratings turning into five-star. A true 4.7 needs about 24,000 extra 5-star ratings. A displayed change is reachable, but whether it happens depends on inflow of new ratings, which public data does not show.

Honest limits

Say these yourself before an interviewer finds them. It builds trust.

1. **Biased sample.** Public text reviews come from a self-selected group. They show symptoms, not causes, and cannot name the employer, city or vendor.
2. **No MoveInSync data.** I have not seen their code, telemetry or customers. Every root cause is a hypothesis with a named test, and effort estimates are guesses.
3. **Approximate counts.** The classifier has precision 0.76 and recall 0.65, about 14% under-count in aggregate, individual categories either way. Confidence intervals cover sampling error only.
4. **Labels were AI-made, with a small human check.** The 30-review human check agreed strongly with the AI labels (kappa 0.84), but 30 is small, and 13 of my 30 labels were matched to official ids from a table the AI suggested.
5. **Below first place, the ranking is a tie.** Only B1 is robust.
6. **Time confound.** A version flagged as bad may simply coincide with a bad month.
7. **Not tested:** collection in Hindi (`lang=hi`), and the optional LLM layer against a real model.
8. **No star-lift forecast.** There is no defensible model for one.
9. **The scoring judgements are mine.** Severity, confidence, controllability and effort are judgement, labelled as such.

Interview questions with answers

Answer in your own words. The short answers below are the spine. Add one example from the data.

The big picture

1. Why are the app's Play Store ratings low? The listing shows 4.6, but 70.5% of the shortfall from 5.0 comes from 1-star ratings, and about one in four recent text reviews is 1 star. The largest complaint group is the app crashing, not opening or loading slowly (about 18% of low-star reviews), followed by login and OTP problems, tracking, and booking failures. Public data shows symptoms, not causes, so the causes are hypotheses to be tested.

2. What would you fix first? First a one-to-two-week instrumentation gate (B0) to get crash, OTP and location data, then stability resilience (B1): show the last known trip with its age, add timeouts and a visible Retry. It is the largest group and the only item that stays in the top three in 79% of 5,000 sensitivity trials.

3. Why not just start building B1 immediately? Because some stability complaints, such as "will not open" and crashes, need crash logs to diagnose. B1's network and loading mitigations can start in parallel with B0, but I would not claim they fix everything.

4. Why do text reviews average 3.8 but the listing 4.6? Different groups. Only about 20% of raters write text, and unhappy users write more often. I cannot prove the cause from public data, but selection bias is the likely explanation.

5. Can you promise the rating will go up? No. I make no star-lift forecast, because there is no defensible model. I can say what it would take to display 4.7 (about 4,000 extra 5-star ratings, if the display is the plain rounded mean), and that fixes should be judged by the failure they target.

Data and method

6. Where did the data come from and can you trust it? Public Google Play reviews via the open-source `google-play-scraper`: 24,314 reviews, 103% of the listing's text-review count, no duplicates or rejects. The scraper can silently stop early, which is why I compared against the listing count.

7. Why only the last year? The app changes over time, so old complaints may no longer apply. The window is 2025-10-10 to 2026-10-09: 2,856 reviews, 846 rated 1-3 stars.

8. How did you decide on 26 categories? I read 502 development reviews and grouped complaints by what people actually wrote, rather than starting from a preset list. One review can have several categories.

9. Why a rule-based classifier instead of an LLM? Free, repeatable, explainable, and no paid API. The weakness is lower recall. The repo includes an optional local-LLM layer, off by default and not measured against a real model.

10. How accurate is the classifier? On a frozen test of 100 fresh reviews: precision 0.76, recall 0.65, F1 0.70. A separate human check on 30 reviews gave F1 0.67. So counts are approximate, and I say so wherever I quote them.

11. What is precision versus recall, in one sentence each? Precision: of what I flagged, how much was right. Recall: of what was really there, how much I found.

12. Who labelled the validation data? First the AI assistant, which is a weakness I disclosed. Then I labelled 30 reviews blind. Kappa was 0.84 against the AI labels, which is strong, but 30 is a small sample and 13 labels were matched to official ids from a suggested table, so I call it a spot check.

13. What did you do about statistical noise? Wilson confidence intervals, two-proportion z-tests, Poisson spike detection with a Bonferroni correction over 230 tests. That correction removed a fake "June 2026 cluster" that was only a volume effect.

14. What is a p-value of 0.73 telling you? Comparing the last 90 days with the previous 90, a difference in mean stars this size would happen by chance about 73% of the time. So there is no distinguishable change.

Prioritisation and recommendation

15. Explain your scoring formula. Reach x Severity x Confidence x Controllability / Effort. Reach is measured from the recent half of the window. The other four are judgement and are labelled as such.

16. Is the ranking reliable? Only first place. I shook the judgement scores and the reach 5,000 times: B1 was in the top three in 79% of trials, the next four items 32 to 45%. I say do not defend the order below the first.

17. Why is B3 ahead of B8 even though B8 scored higher? By judgement, not by score: B3 blocks use of the app, touches a safety case, and its OTP part is stable over time. B8′s high score comes from a guessed small effort, and its reach is only 5.4%.

18. Why does operations (B9) have high reach but sit last? The app team cannot control it. It is a Customer Success matter, so I hand the evidence to account owners per client.

19. What is the instrumentation gate (B0)? Pull crash and ANR data by version and device, the OTP funnel, location ping gaps, no-show events, and Play Console ratings by version. It turns each root-cause hypothesis into a confirmed or rejected cause.

20. How would you know a fix worked? By internal telemetry for the targeted failure (crash rate, OTP success, forced logouts), compared between old and new versions in a staged rollout on the same days. Public complaint rates are a slow secondary check.

21. Why can't Play reviews measure a release? With about 240 text reviews a month, only a swing of roughly 11 points in the 1-star share is detectable. Normal month-to-month movement is already that large.

Ethics and honesty

22. Is there a way to raise the rating faster? Only honest ones: fix the causes, reply to reviews usefully, and use the official in-app review prompt for everyone. I rule out incentives, review gating, steering users to 5 stars, or hiding negative reviews.

23. This looks AI-generated. Did you do it yourself? I used Claude to help plan, research, code, test and draft, and I disclose that in the repo. I made the scope decisions, ran everything on my machine, reproduced the numbers, and did the human labelling check. I can walk through any file, and I can tell you what the first version got wrong. (Say only what is true for you.)

24. What was wrong with your first version? The independent review found: a June 2026 cluster that was just a volume artifact, rating maths using the rounded 4.6 instead of 4.6377, an inflated weight for a faded login burst, and some Help Center claims not yet in my source register. I fixed all of them.

25. What are the biggest limits? Biased sample, no MoveInSync data, approximate counts, tie in the ranking below first place, and AI-made labels with only a 30-review human check.

The tool

26. How do you answer "How are our Play Store ratings?" each month? Run `.\scripts\run_all.ps1`. It cleans the new snapshot, prints the headline (last 90 days against the previous 90, with a confidence interval), and writes the full report with themes, spikes and a comparison to the previous snapshot.

27. How do you know the tool is correct? 85 automated tests, including a golden check that quoted numbers match the data, a taxonomy hash check, and a CI workflow on GitHub.

28. What would you add with more time? Collect with `lang=hi`, measure the optional LLM layer against a real model, a larger independent human audit, and join the findings to internal telemetry when available.

Curveballs

29. What would you do in your first week at MoveInSync? Run B0 with Engineering, get Play Console ratings by version, open the top three tickets with real effort estimates, and ask Customer Success which clients drive the operations complaints.

30. What if MoveInSync says the app is fine and the drivers are the problem? Some of it is: operations complaints (late cabs, routing) are about 15% of the pool and I hand them to Customer Success. But the app itself still shows measurable stability and login complaints, and B0 would settle it with data.

Demo script and 2-minute pitch

The 2-minute pitch (say it in your own words)

"MoveInSync's employee app shows 4.6 stars on Google Play, but one in four recent text reviews is a 1-star review. I wanted to know why, using only public data.

I collected about 24,000 public reviews, kept the last year, and sorted 846 low-star reviews into 26 complaint types with a rule-based classifier. I tested its accuracy on fresh reviews: precision 0.76, recall 0.65. I also checked 30 labels myself, and they agreed strongly with the earlier AI labels.

The biggest group, about 18%, is the app crashing, not opening or loading slowly. I ranked fixes with a scoring model and shook it 5,000 times: only stability came out robustly first. So I recommend a short instrumentation step to get real crash and OTP data, then a stability-resilience fix, with login clarity and a skippable rating prompt next.

I do not promise a star lift. Public reviews are a biased sample, and fixes should be judged with internal telemetry. And because this question will be asked again, I built a tool, with 85 tests, that answers it with one command."

Live demo (about 2 minutes)

1. Open PowerShell in the project folder and activate the environment: `.\.venv\Scripts\Activate.ps1`
2. Run `.\scripts\run_all.ps1`. It picks the newest snapshot, cleans it, prints the headline, and writes the report.
3. Point at the headline: "Last 90 days 3.8 stars, 25% one-star, no statistically distinguishable change from the previous 90 days."
4. Open `reports\analysis_<run id>\report.md` and show section 0 ("How are our Play Store ratings?"), then the themes table.
5. Run `python -m pytest` and show 85 passed.
6. Open `docs/EXECUTIVE_RECOMMENDATION.md` and `docs/engineering_tickets.md` to show what Engineering would receive.

Before the interview

- Run `run_all.ps1` once the day before so you have seen the output.
- Read `docs/EXECUTIVE_RECOMMENDATION.md` and `docs/methodology.md` once, slowly.
- Practise answers 1, 2, 12, 16, 22 and 23 out loud. Those are the ones most likely to be asked.
- Open your GitHub repo in a private window to confirm it is public.